

# CSE 406: Computer Security Sessional Project Design Report

## Off-Path TCP Reset Attack on Telnet and In-Line Gateway Defense

### C2 Group 6

Ariful Islam Shadhin 2105166

Mustafa Muhaimin 2105178

## 1 Overview of the Project Idea

### 1.1 Project Context and Motivation

Although unencrypted Telnet (RFC 854) is retired in modern IT, it remains prevalent in Operational Technology (OT) networks. Legacy Remote Terminal Units (RTUs) and PLCs rely on Telnet because constrained microcontrollers cannot support SSH cryptographic handshakes without stalling real-time control loops.

Crucially, legacy embedded TCP stacks (e.g., lwIP, older VxWorks) implement baseline RFC 793 without RFC 5961 Challenge-ACK safeguards. Any TCP segment with the RST flag set is accepted if its sequence number falls anywhere inside the receive window:  $RCV.NXT \leq SEG.SEQ < RCV.NXT + W$ . This allows an off-path attacker—with zero packet visibility—to terminate established sessions without brute-forcing the full 32-bit sequence space.

### 1.2 Targeted Vulnerabilities

Our attack exploits three compounded vulnerabilities across protocol, implementation, and operations:

1. **RFC 793 In-Window Acceptance:** With a 64 KB receive window ( $W = 65,536$ ), the candidate space shrinks from  $2^{32}$  to  $2^{32}/W = 65,536$  slots.
2. **Predictable Embedded ISN Clocks:** Many embedded stacks advance ISNs using linear timer increments rather than RFC 6528 PRNGs. Sampling port 23 via a legitimate handshake reveals the server's sequence clock baseline.
3. **SCADA Quiescent Intervals:** Periodic polling creates 2–10 s idle pauses between queries where  $\Delta SEQ = 0$ , holding the target receive window stationary.

### 1.3 Attack Architecture: Two-Phase Off-Path Exploitation

Instead of high-rate packet flooding that triggers perimeter alarms, we execute a two-phase attack:

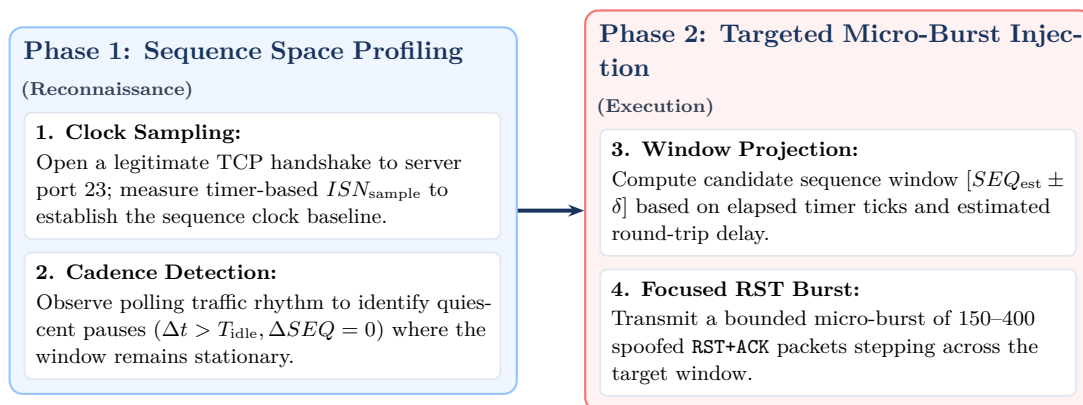


Figure 1: Two-Phase Attack Design: Non-Destructive Sequence Profiling → Focused Micro-Burst Reset.

## 2 Network Topology, Components and Protocol Sequence

### 2.1 Network Topology and Testbed Architecture

Our experimental testbed models an industrial network with two isolated subnets interconnected by an intermediate Linux router. The entire environment is deployed using containerized Docker nodes:

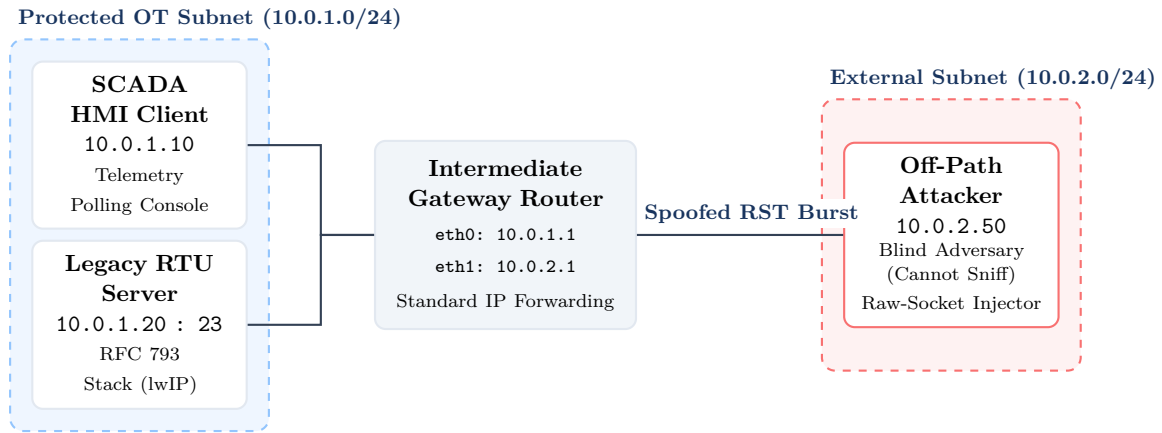


Figure 2: Routed Testbed Topology: Subnet Isolation and Intermediate Gateway Router.

- **SCADA HMI Client (10.0.1.10)**: Industrial operator console that periodically polls RTU registers over Telnet.
- **Legacy RTU Server (10.0.1.20:23)**: Embedded controller implementing baseline RFC 793 ( $W = 65,535$ ).
- **Intermediate Gateway Router (10.0.1.1 / 10.0.2.1)**: Dual-homed Linux router providing standard packet forwarding between subnets.
- **Off-Path Attacker (10.0.2.50)**: Out-of-band adversary capable of injecting spoofed IP packets across the router.

### 2.2 Protocol Timing and Interaction Flow

Figure 3 illustrates the complete attack execution sequence against an undefended legacy RTU, showing how clock reconnaissance, quiescent interval detection, and in-window reset injection successfully tear down the active session:

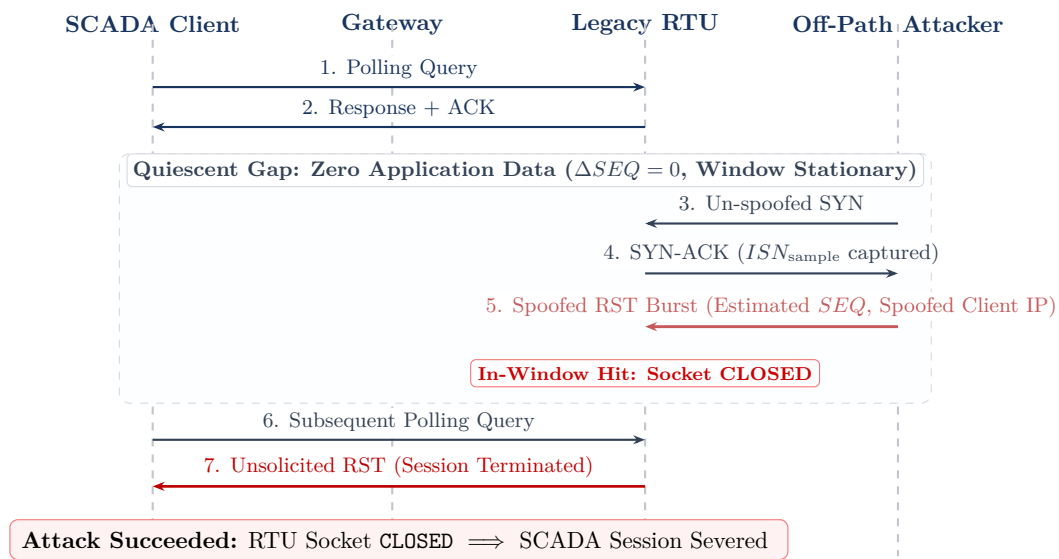


Figure 3: Protocol Timing Sequence: Reconnaissance Sampling, Quiescent Burst Injection, and Connection Teardown.

### 3 Packet and Frame Structure

#### 3.1 Low-Level Packet Construction and Serialization

All attack packets are crafted from scratch at the raw byte level using Python raw sockets (`AF_INET`, `SOCK_RAW`) with manual header inclusion (`IP_HDRINCL`). We serialize the standard 20-byte IPv4 header and 20-byte TCP header directly into network byte order (big-endian), strictly without high-level packet manipulation libraries.

Tables 1 and 2 illustrate the exact 32-bit field layouts:

| IPv4 Header (20 Bytes — 160 Bits Total)               |   |                        |   |                                                  |    |                              |    |
|-------------------------------------------------------|---|------------------------|---|--------------------------------------------------|----|------------------------------|----|
| Bit 0                                                 | 3 | Bit 4                  | 7 | Bit 8                                            | 15 | Bit 16                       | 31 |
| Version: 4 (4b)                                       |   | IHL: 5 (4b)            |   | DSCP / ECN: 0x00 (8b)                            |    | Total Length: 40 Bytes (16b) |    |
| Identification: 0x1337 (16b)                          |   |                        |   | Flags: 0x02 [DF] (3b) & Fragment Offset: 0 (13b) |    |                              |    |
| TTL: 64 (8b)                                          |   | Protocol: 6 [TCP] (8b) |   | Header Checksum: Calculated RFC 1071 (16b)       |    |                              |    |
| Source IP Address: 10.0.1.10 (Spoofed Client IP)      |   |                        |   |                                                  |    |                              |    |
| Destination IP Address: 10.0.1.20 (Target RTU Server) |   |                        |   |                                                  |    |                              |    |

Table 1: 32-bit IPv4 Header Breakdown: Raw Big-Endian Serialization.

| TCP Header (20 Bytes — 160 Bits Total)                |   |                  |   |                                     |    |                      |    |
|-------------------------------------------------------|---|------------------|---|-------------------------------------|----|----------------------|----|
| Bit 0                                                 | 3 | Bit 4            | 7 | Bit 8                               | 15 | Bit 16               | 31 |
| Source Port: Ephemeral (16b)                          |   |                  |   | Destination Port: 23 [Telnet] (16b) |    |                      |    |
| Sequence Number: In-Window Candidate <i>SEQ</i> (32b) |   |                  |   |                                     |    |                      |    |
| Acknowledgment Number: Echoed from sampling / 0 (32b) |   |                  |   |                                     |    |                      |    |
| Data Offset: 5 (4b)                                   |   | Reserved: 0 (3b) |   | Flags: 0x14 (RST+ACK)               |    | Window Size: 0 (16b) |    |
| TCP Checksum: Pseudo-Header Sum (16b)                 |   |                  |   | Urgent Pointer: 0 (16b)             |    |                      |    |

Table 2: 32-bit TCP Header Breakdown: Exploit Parameters vs. Transport Fields.

#### 3.2 Modified Header Fields and Technical Rationale

Table 3 details the specific packet header fields manipulated during the attack, distinguishing between active exploit variables and mandatory transport integrity requirements:

| Layer | Field      | Value Assigned       | Technical Purpose and Engineering Rationale                                                                                                                                                       |
|-------|------------|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| IPv4  | Source IP  | 10.0.1.10            | <b>Exploit (Spoofed):</b> Impersonates the legitimate SCADA operator to pass routing and socket matching.                                                                                         |
| IPv4  | Checksum   | RFC 1071 Sum         | <b>Integrity:</b> Standard IP checksum required for intermediate router forwarding.                                                                                                               |
| TCP   | Dest Port  | 23 (Telnet)          | <b>Targeted:</b> Directs the segment to the listening Telnet daemon on the RTU.                                                                                                                   |
| TCP   | Seq Number | In-window <i>SEQ</i> | <b>Exploit:</b> Placed within $[RCV.NXT, RCV.NXT + W)$ to satisfy RFC 793 acceptance.                                                                                                             |
| TCP   | Flags      | 0x14 (RST+ACK)       | <b>Exploit:</b> Includes ACK flag to bypass stateful firewalls expecting bidirectional traffic.                                                                                                   |
| TCP   | Checksum   | Pseudo-Header Sum    | <b>Integrity:</b> Manually calculated using the <b>spoofed Source IP</b> in the 12-byte pseudo-header. Without this, the target's TCP stack drops the packet before checking the sequence number. |

Table 3: Summary of Header Fields: Active Exploit Parameters vs. Protocol Compliance.

## 4 Project Work Plan

The project is structured into five sequential tasks ordered strictly by technical dependencies, taking the system from isolated network setup to full attack verification and in-line defense:

### 1. Isolated Testbed Provisioning and Routing Setup

Build a multi-container Docker Compose environment with two separate subnets (10.0.1.0/24 and 10.0.2.0/24) interconnected by a dual-homed Linux router. Configure routing tables and verify that the attacker container has zero promiscuous packet visibility, ensuring a genuine off-path threat model.

### 2. Raw Packet Crafter and Checksum Engine

Implement a standalone Python raw-socket injection module. Write RFC 1071 one's complement checksum algorithms for both IPv4 headers and the TCP pseudo-header. Verify that handcrafted packets deserialize cleanly and pass checksum validation in Wireshark.

### 3. Reconnaissance: Clock Sampling and Cadence Detection

Build the pre-attack profiling tool. Implement legitimate TCP handshakes to sample the target's timer-based ISN counter and establish a clock progression baseline. Add an automated traffic observer that identifies quiescent intervals ( $\Delta t > T_{\text{idle}}$ ) between SCADA queries.

### 4. Attack Integration and Session Teardown Verification

Combine the sampling telemetry with the raw-socket injector to launch micro-burst tests against an undefended RFC 793 RTU. Confirm session teardown through client-side terminal disconnection, server socket state transition to CLOSED, and subsequent rejection packets.

### 5. In-Line Protective Gateway Defense Implementation

Implement the two-stage defense on the Linux router using `iptables` and Linux `NFQUEUE`. Deploy Stage 1 handshake anomaly rate limits and Stage 2 Delayed Reset Quarantine with active Zero-Window challenge probing, verifying that legitimate sessions remain protected during spoofed reset bursts.

## 5 Expected Outcomes and Observable Behaviors

When our off-path attack executes against an unprotected legacy RTU server, the resulting connection collapse produces clear, verifiable indicators across both endpoints:

- **Client Terminal Disconnection:** The operator's interactive Telnet session terminates abruptly with the standard network socket break notification: `Connection closed by foreign host.`
- **Server Socket State Deallocation:** Diagnostic commands (`ss -t -a` or `netstat`) confirm that the TCP socket on port 23 transitions immediately from ESTABLISHED to CLOSED, releasing all associated socket buffers.
- **Subsequent Packet Rejection:** Any subsequent delayed packet transmitted by the SCADA client receives an immediate unsolicited RST from the RTU, confirming that the session state no longer exists.

Under realistic network conditions, session teardown does not rely on an idealized single-packet hit or a disruptive brute-force sweep. Instead, the combination of timer-based ISN sampling and quiescent interval detection constrains the search space to a bounded micro-burst of **150 to 400 packets**. This achieves an empirical overhead reduction of over **95%** compared to full window brute-forcing ( $2^{32}/W = 65,536$  packets). Because the micro-burst is dispatched during an idle polling window ( $\Delta SEQ = 0$ ), teardown occurs within one local Round-Trip Time ( $< 50$  ms) while remaining well beneath typical perimeter flood detection thresholds ( $\geq 5,000$  packets/sec).

## 6 Defense Idea

### 6.1 Two-Stage In-Line Protective Gateway

Because legacy industrial RTUs cannot be modified or upgraded with RFC 5961 support without expensive recertification, we introduce an **In-Line Protective Gateway** deployed as a transparent bump-in-the-wire appliance between the subnets:

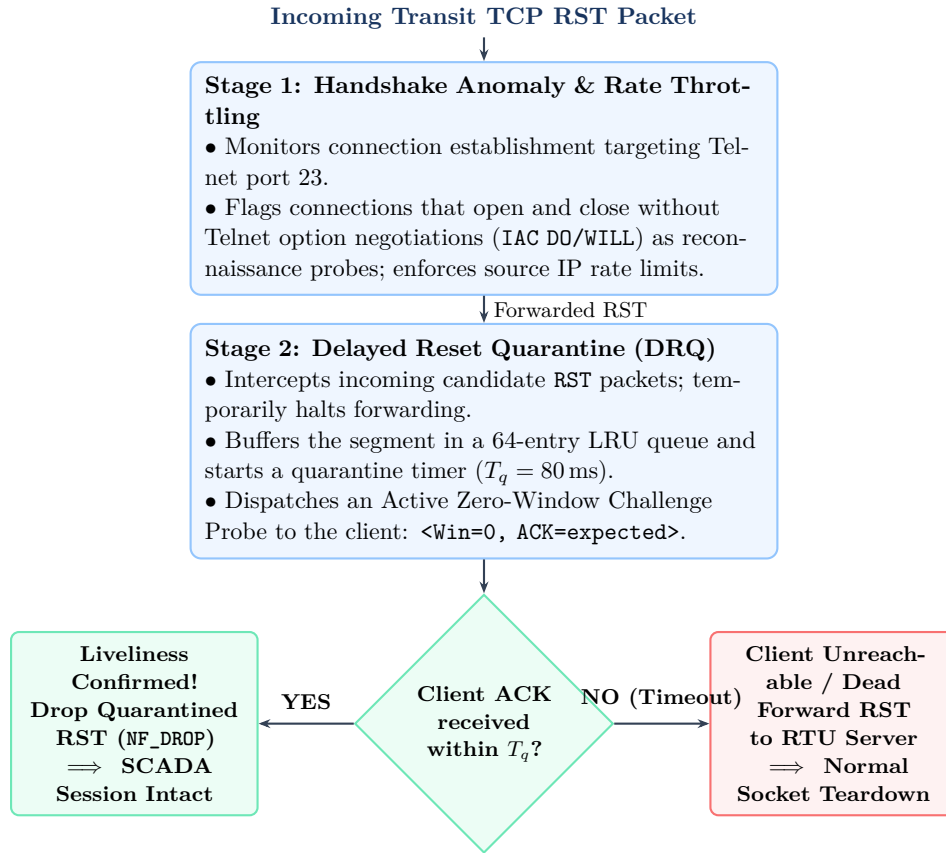


Figure 4: In-Line Protective Gateway Pipeline: Handshake Inspection and Active Challenge Verification.

### 6.2 Operational Robustness and Practical Trade-offs

- **Handling Idle Clients Accurately:** Passive inspection cannot tell whether a quiet client is disconnected or simply idle. By emitting an active Zero-Window Probe ( $\text{Win}=0$ ,  $\text{ACK}=\text{expected}$ ), the gateway forces the legitimate client stack to return an immediate protocol-level ACK without pushing application data, confirming client liveliness in real time.
- **Protection Against State Flooding:** To prevent an attacker from exhausting gateway memory with an RST flood, the quarantine buffer uses a fixed 64-entry ring buffer paired with a token-bucket rate limiter ( $R_{\text{max}} = 50$  evaluations/sec). Packets exceeding this quota are dropped immediately at wire speed.
- **Acceptable Disconnect Latency:** Legitimate operator disconnects experience a minor intentional hold of  $T_q = 80$  ms. In industrial SCADA operations, this sub-100 ms delay is an imperceptible trade-off for deterministic protection against unauthorized connection hijacking and disruption.